Frontend Build Documentation

This page explains how to get the project running.

What you need installed

- Node.js (LTS recommended)
- npm

(Optional)

- Xcode if you want to run on iOS
- Android Studio and the Android SDK if you want to run on Android

Clone the project

```
git clone git@github.com:amosproj/amos2026ss03-ailixir-intelligence.git
cd frontend/ailixir
```

If you already have the workspace open, just make sure your terminal is inside `frontend/ailixir`.

Install dependencies

```
npm install
```

This installs the JavaScript dependencies needed for Expo, Tamagui, Firebase, and the app screens under `src/app/`.

Start the app locally

```
npx expo start -c
```

You can also use the package script:

```
npm run start
```

Use the Expo developer tools to open the app on a simulator, emulator, or physical device.

- Android emulator
- iOS simulator

Build and verify the code

Before sharing changes, run the checks the repository expects:

```
npx tsc --noEmit
npm run lint
```

For platform targets, use:

```
npm run web
npm run ios
npm run android
```

Learn more

To learn more about developing your project with Expo, look at the following resources:

- Expo documentation: Learn fundamentals, or go into advanced topics with our guides.
- Learn Expo tutorial: Follow a step-by-step tutorial where you'll create a project that runs on Android, iOS, and the web.

Backend Build Documentation

How to set up, run, and deploy the Ailixir backend (API + worker).

The backend is two Python services on Google Cloud Run, fronted by infrastructure provisioned through Terraform. **Deployment is fully automated:** every push to `main` builds containers, plans Terraform, and updates Cloud Run via GitHub Actions. The sections below cover one-time setup, local development, and the manual deployment path used for hotfixes.

Prerequisites

Tool	Version	Why
Python	3.11	Both services target 3.11
Docker	recent	Container builds (only required for the deploy path)
Terraform	1.5+	Manual `terraform apply` (only required for the manual path)
`gcloud` CLI	recent	Application Default Credentials for local development
Git	any	Clone and contribute
GCP project	—	A Firebase / GCP project with billing enabled (default ID: `amos26`)

One-time setup

These steps create the external resources Terraform expects to already exist. They are done once per environment.

1. Provision Neo4j

The knowledge graph stage uses Neo4j. Terraform does **not** create the cluster — you must supply a reachable URI, user, and password. Recommended path:

- Create a free Neo4j Aura instance at https://console.neo4j.io.
- Note the **Connection URI** (e.g. `neo4j+s://xxxxxxxx.databases.neo4j.io`), the **username** (`neo4j`), and the **password** shown only at creation time.

2. Create a Document AI processor

OCR uses Google Cloud Document AI. Terraform enables the API but does **not** create the processor itself.

- In GCP Console → **Document AI** → **Create Processor**.
- Pick **Form Parser** — this populates `extracted_fields` with key-value pairs that the knowledge graph uses. (The "Document OCR" processor type works too but only fills `raw_text_blocks`, leaving `extracted_fields` empty.)
- Set the region (e.g. `us`).
- Note the **Processor ID**.

3. Create the Terraform state bucket

State for both services is stored in `gs://amos2026ss03-ailixir-tf-state` (see `backend.tf` in each terraform directory). Create the bucket once:

```
gcloud storage buckets create gs://amos2026ss03-ailixir-tf-state \
  --location=us-east1 --uniform-bucket-level-access
gsutil versioning set on gs://amos2026ss03-ailixir-tf-state
```

4. Configure GitHub Secrets

The CI/CD pipeline needs the following repository secrets (Settings → Secrets and variables → Actions):

Secret	Used by	Value
`GCP_PROJECT_ID`	API + Worker	Your GCP project ID (e.g. `amos26`)
`GCP_SA_KEY`	API + Worker	JSON key for a service account with `roles/run.admin` and `roles/storage.admin`
`NEO4J_URI`	Worker only	From step 1

Secret	Used by	Value
NEO4J_USER	Worker only	From step 1
NEO4J_PASSWORD	Worker only	From step 1
DOCUMENT_AI_PROCESSOR_ID	Worker only	From step 2

5. Get a Firebase service account key (local development only)

For local dev you need a service-account JSON key so the SDKs can authenticate to Firebase/Firestore:

- Firebase Console → **Project Settings** → **Service Accounts** → **Generate new private key**.
- Save the file at `Backend/secrets/serviceAccountKey.json` (gitignored).

In production this is not used — Cloud Run uses Application Default Credentials via the attached service account.

Local development

Setup

```
git clone git@github.com:amosproj/amos2026ss03-ailixir-intelligence.git
cd amos2026ss03-ailixir-intelligence/Backend

python -m venv .venv
source .venv/bin/activate

# Install API dependencies
pip install -r api/requirements.txt
# Or, for worker work:
pip install -r workers/requirements.txt

# Copy and edit env file
cp .env.example .env
# fill in FIREBASE_KEY_RELATIVE_PATH, NEO4J_*, VERTEX_*, etc.
```

Authenticate ADC once for the Vertex AI calls used by the worker:

```
gcloud auth application-default login
```

Run the API locally

```
cd Backend
uvicorn api.main:app --reload
```

- Health: http://127.0.0.1:8000/health
- Interactive API docs (Swagger): http://127.0.0.1:8000/docs
- OpenAPI spec: http://127.0.0.1:8000/openapi.json

Run the worker locally

The worker normally only receives traffic from Pub/Sub. To run it locally without Pub/Sub authentication:

```
cd Backend
export PUBSUB_SKIP_OIDC_VERIFICATION=1
export SKIP_STARTUP_CONNECTIONS=1    # skips Neo4j connect on startup if you only
need the HTTP layer
uvicorn workers.main:app --reload --port 8080
```

To exercise the pipeline end-to-end, omit `SKIP_STARTUP_CONNECTIONS` (Neo4j must be reachable) and `POST` a Pub/Sub-shaped envelope to `http://127.0.0.1:8080/pubsub/push`.

Deployment

Automatic (the normal path)

Pushing to `main` triggers the GitHub Actions pipelines:

- `.github/workflows/backend-ci-cd.yml` — fires when `Backend/api/**` changes
- `.github/workflows/worker-ci-cd.yml` — fires when `Backend/workers/**` changes

Each pipeline does the same five steps:

1. Lint with flake8.
2. Verify the app starts (10-second smoke).
3. Build the Docker image, tag with the commit SHA, push to `gcr.io/<project>/ailixir-{backend|worker}:<sha>`.
4. `terraform init` + `terraform plan` + `terraform apply` against the per-service state prefix.
5. Cloud Run picks up the new image; the previous revision keeps serving until the new one is healthy.

PRs run CI only — the deploy job is gated on `github.ref == 'refs/heads/main'`.

Manual Terraform apply (rare)

Use this only when you need to apply Terraform changes without bumping the image — for example, adjusting IAM or env vars on an existing deploy.

Requires `gcloud auth application-default login` first, and your account must have the IAM roles listed in *Configure GitHub Secrets* above (`roles/run.admin`, `roles/storage.admin`, plus the API enablement

permissions Terraform implicitly needs).

```
cd Backend/api/terraform        # or Backend/workers/terraform
terraform init -reconfigure
terraform plan \
  -var="project_id=amos26" \
  -var="image_tag=latest"
terraform apply \
  -var="project_id=amos26" \
  -var="image_tag=latest"
```

Worker apply additionally requires the Neo4j and Document AI variables:

```
terraform apply \
  -var="project_id=amos26" \
  -var="image_tag=latest" \
  -var="neo4j_uri=..." \
  -var="neo4j_user=..." \
  -var="neo4j_password=..." \
  -var="document_ai_processor_id=..."
```

What gets deployed

Component	Type	Name	Region
API	Cloud Run service	`ailixir-backend`	us-east1
Worker	Cloud Run service	`ailixir-worker`	us-east1
Documents bucket	GCS bucket	`ailixir-documents-amos26`	us-east1
Cypher exports bucket	GCS bucket	`ailixir-cypher-amos26`	us-east1
Event topic	Pub/Sub topic	`document-uploaded`	global
Dead-letter topic	Pub/Sub topic	`document-uploaded-dlq`	global
Subscription	Pub/Sub push subscription	`document-uploaded-ingestion`	global
API service account	IAM SA	`ailixir-api@<project>`	—
Worker service account	IAM SA	`ailixir-worker@<project>`	—
Pub/Sub pusher SA	IAM SA	`ailixir-pubsub-pusher@<project>`	—

Component	Type	Name	Region
Firestore indexes	Firestore composite indexes	(5 indexes on `documents`)	same as Firestore database

Firestore indexes are deployed separately (not via Terraform) using the Firebase CLI:

```
firebase deploy --only firestore:indexes --project amos26
```

The index definitions live in `Backend/firestore.indexes.json` and `firebase.json` at the repo root tells the CLI where to find them.

Verification

After a deploy:

1. **Health check** — `curl https://<backend-url>/health` should return `{"status":"ok"}`.
2. **OpenAPI loads** — open `https://<backend-url>/docs` and confirm Swagger UI renders.
3. **Worker health** — `curl https://<worker-url>/health` (the worker is publicly reachable but its `/pubsub/push` route requires OIDC).
4. **Logs** — Cloud Logging filtered by the Cloud Run service name shows structured logs with `request_id` (API) or `message_id` (worker).
5. **Test an endpoint** — follow the Document API Frontend Integration Guide for ready-to-paste `curl` commands and how to get a Firebase ID token.

Troubleshooting

Symptom	Likely cause
`FileNotFoundError: Firebase credentials file not found` locally	`FIREBASE_KEY_RELATIVE_PATH` is set but the file doesn't exist at `Backend/<that path>` — either fix the path or unset the env var to fall back to ADC.
Worker `KeyError: 'NEO4J_URI'` on startup	The Neo4j env vars weren't passed to Terraform; check the GitHub Secrets and re-run the deploy.
API `ACCESS_TOKEN_SCOPE_INSUFFICIENT` when generating signed URLs	The IAM Credentials API isn't enabled, or the `roles/iam.serviceAccountTokenCreator` self-binding is missing — both are in Terraform; re-apply.
Worker returns 401 to Pub/Sub	The OIDC token's `email` claim doesn't match `PUBSUB_PUSH_SERVICE_ACCOUNT` — most often because terraform was applied in a way that recreated the pusher SA. Re-apply `workers/terraform`.
Documents stuck in `pending_upload`	The client never called `POST /documents/{id}/finalize`, or the underlying GCS objects don't exist (the API HEAD-

Symptom	Likely cause
	checks them before transitioning).
Worker successfully receives event but pipeline silently fails	Check Cloud Logging for the worker — most often `DOCUMENT_AI_PROCESSOR_ID` is missing or wrong.

Include the `X-Request-ID` header value (returned on every API response) when filing a bug — backend engineers grep Cloud Logging by it.